

# Poverty Prediction and Regional Forecasting in Armenia

---

Machine Learning on ILCS 2015 Household Survey Data  
and ArmStat Regional Panel (2016–2022)

**Author** Hovo Sukiasyan

**Supervisor** Aleksandr Hayrapetyan

**Program Chair** Hayk Nersisyan

**Institution** American University of Armenia

**Date** May 2026

## Abstract

Poverty measurement in Armenia relies heavily on periodic household surveys and annual regional estimates, leaving a gap between data collection cycles and timely policy decisions. This project addresses that gap in two strands: supervised machine learning on ILCS 2015 to predict household income, and regional time-series forecasting on ArmStat statistics for Armenia’s eleven marzes (2016–2022).

The most informative regional signal is available at **annual** (survey-based) resolution; neural sequence models benchmarked alongside Lag-1, Ridge autoregressive, and ARIMA baselines systematically **underperform** Lag-1 on yearly poverty rate because each marz supplies only seven true annual training points (§5). Auxiliary **monthly** and **daily** views are obtained by forward-filling annual poverty rates; at those granularities goodness-of-fit is almost mechanically inflated because very little genuine movement occurs within survey years (§5).

Out-of-sample **credibility** is established by a strict **2022 holdout backtest**: models are trained exclusively on 2016–2021 data and scored against published 2022 rates. Lag-1 attains the best regional mean absolute error (5.07 percentage points); a 50/50 Lag-1/Ridge ensemble attains nearly zero population-weighted national error (0.02 percentage points).

Results and 2023–2026 projections are communicated through an interactive Next.js dashboard. This document is the project’s primary technical reference on data, methodology, architecture, and findings.

Document overview

**Sections 1–3** cover the project framing, methodology, and system architecture. **Section 4** describes both datasets and key descriptive statistics. **Section 5** presents all model results, including the primary 2022 holdout backtest. **Section 6** documents the interactive web application. **Sections 7–9** address challenges, future work, and conclusions. **Appendix A** lists data and code locations for reproducibility.

Contents

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                    | <b>4</b>  |
| 1.1      | System Overview . . . . .                              | 4         |
| 1.2      | Objectives . . . . .                                   | 4         |
| 1.3      | Scope . . . . .                                        | 5         |
| <b>2</b> | <b>Methodology</b>                                     | <b>6</b>  |
| 2.1      | Literature Context and Motivation . . . . .            | 6         |
| 2.2      | Data Sources . . . . .                                 | 6         |
| 2.3      | Feature Engineering and Preprocessing . . . . .        | 7         |
| 2.4      | Validation Strategy . . . . .                          | 7         |
| <b>3</b> | <b>System Architecture</b>                             | <b>8</b>  |
| 3.1      | Overview . . . . .                                     | 8         |
| 3.2      | Technology Stack . . . . .                             | 8         |
| <b>4</b> | <b>Data Description and Schema</b>                     | <b>10</b> |
| 4.1      | ILCS 2015 Dataset . . . . .                            | 10        |
| 4.2      | ArmStat Regional Panel . . . . .                       | 10        |
| 4.3      | Key Descriptive Statistics . . . . .                   | 11        |
| 4.4      | Seasonal Income Variation . . . . .                    | 11        |
| <b>5</b> | <b>Model Development and Results</b>                   | <b>13</b> |
| 5.1      | Household Income Models . . . . .                      | 13        |
| 5.2      | Feature Importance . . . . .                           | 13        |
| 5.3      | Bayesian Hyperparameter Optimization . . . . .         | 14        |
| 5.4      | Time-Series Analysis — Classical Models . . . . .      | 15        |
| 5.5      | Time-Series Analysis — Neural Network Models . . . . . | 16        |
| 5.6      | 2022 Forecast Validation (Primary Evidence) . . . . .  | 17        |
| <b>6</b> | <b>Application and Functionality</b>                   | <b>21</b> |
| 6.1      | Homepage . . . . .                                     | 21        |
| 6.2      | Regional Explorer . . . . .                            | 21        |
| 6.3      | Household Income Model . . . . .                       | 23        |
| 6.4      | Feature Importance Page . . . . .                      | 23        |

---

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 6.5      | Forecasting Diagnostics . . . . .             | 24        |
| 6.6      | 2022 Forecast Validation . . . . .            | 24        |
| 6.7      | Future Projection 2023–2026 . . . . .         | 26        |
| <b>7</b> | <b>Challenges Encountered</b>                 | <b>28</b> |
| 7.1      | Data Quality and Panel Size . . . . .         | 28        |
| 7.2      | Anomalous Regional Observations . . . . .     | 28        |
| 7.3      | Interview Month Recovery . . . . .            | 28        |
| 7.4      | Web Application Data Architecture . . . . .   | 28        |
| 7.5      | Model Evidence Communication . . . . .        | 28        |
| <b>8</b> | <b>Future Work</b>                            | <b>29</b> |
| 8.1      | Larger Regional Panels . . . . .              | 29        |
| 8.2      | Improved Household Income Models . . . . .    | 29        |
| 8.3      | Scenario Forecasting . . . . .                | 29        |
| 8.4      | Confidence Intervals on Projections . . . . . | 29        |
| 8.5      | ILCS Longitudinal Analysis . . . . .          | 29        |
| <b>9</b> | <b>Conclusion</b>                             | <b>31</b> |
| <b>A</b> | <b>Data and Code Availability</b>             | <b>32</b> |
| <b>B</b> | <b>*</b>                                      | <b>33</b> |
| <b>C</b> | <b>*</b>                                      | <b>33</b> |

# 1 Introduction

---

## 1.1 System Overview

The project is a full-stack data science system for predicting and forecasting household poverty in Armenia. It combines two data assets — a nationally representative household survey (ILCS 2015) and a seven-year regional panel from the Armenian Statistical Committee (ArmStat) — with a layered modelling pipeline and an interactive web interface.

At the micro level, cross-sectional ML models on ILCS 2015 identify which household characteristics (income source diversification, consumption expenditures, asset ownership) are most predictive of total household income. At the macro level, time-series models on the ArmStat panel estimate the future trajectory of regional poverty rates and a composite socioeconomic stress index, with credibility established through a 2022 holdout backtest trained entirely on 2016–2021 data.

The research findings and forecasts are exposed through a Next.js web application (<https://armenia-poverty.vercel.app>) that allows users to explore regional trends interactively, inspect model performance metrics, review the 2022 validation evidence, and examine 2023–2026 poverty projections. A PostgreSQL (Neon) database supports the dynamic data features, while validation results and forecast outputs are served as static CSV files for reproducibility. Data artefacts, scripts, and deployment details are summarised in Appendix A.

## 1.2 Objectives

The primary objectives of this project are as follows.

1. Identify the household-level features most predictive of total income in Armenia using supervised machine learning on ILCS 2015 data, and quantify model performance through rigorous train/test evaluation with Bayesian hyperparameter optimization.
2. Construct and validate autoregressive and neural time-series models that forecast regional poverty rates across Armenia’s 11 marzes, using a strict held-out 2022 backtest for credibility assessment.
3. Build and deploy a public-facing web dashboard that communicates results, spatial patterns, and forecasts in an accessible, interactive format suitable for researchers and policy practitioners.
4. Document the complete data pipeline, modelling workflow, and web system architecture so that the project can be reproduced and extended.

A secondary objective is methodological: to honestly assess the limitations of poverty prediction with small regional panels and cross-sectional survey data, and to communicate uncertainty clearly rather than overstate model capability.

### 1.3 Scope

The project covers the following scope.

**Data:** (i) ILCS 2015 public-use household microdata — 5,184 households, 29 research columns after cleaning; (ii) ArmStat regional statistics — 11 marzes, 7 annual observations per marz (2016–2022), disaggregated to monthly frequency for time-series modelling.

**Models:** Five supervised regressors (Random Forest, Gradient Boosting, Extra Trees, Ridge, Lasso) with and without Bayesian hyperparameter optimization; four neural sequence models (GRU, BiLSTM, Transformer, TCN) trained at yearly, monthly, and daily granularity; and classical baselines (Lag-1, Ridge AR, ARIMA).

**Web application:** A Next.js/React web app hosted on Vercel, with pages for the homepage, regional explorer, regional trends, regional rankings, household model, feature importance, forecasting diagnostics, 2022 validation, and future projections.

**Out of scope:** Real-time data ingestion, mobile clients, causal inference, and individual-level poverty status classification (as opposed to income regression).

## 2 Methodology

The project follows an applied quantitative research design combining secondary data analysis, supervised machine learning, and time-series forecasting. All models are evaluated against held-out test data; no model is assessed only on training performance.

### 2.1 Literature Context and Motivation

Poverty measurement in low- and middle-income countries often relies on proxy means tests (PMT): scores based on observable characteristics when direct income measurement is imperfect or costly [1]. Machine-learning extensions of welfare scoring have demonstrated that ensemble trees and gradient boosting can outperform simple linear scores out of sample on survey data [2]. Absolute predictive power nonetheless remains modest in many applications, which is consistent with an  $R^2$  of order 0.35 for household income on a single ILCS cross-section in this project.

For regional poverty forecasting, autoregressive models remain competitive with neural alternatives on short panels [3]. Armenia’s annual regional data (7 observations per marz) is at the lower bound for stable neural network training, and the results confirm that Lag-1 and Ridge AR outperform GRU and LSTM on the annual benchmark, while neural models are competitive only at monthly granularity where the data is synthetically interpolated.

### 2.2 Data Sources

#### *ILCS 2015 Household Survey*

The Integrated Living Conditions Survey (ILCS) is Armenia’s primary household welfare instrument, conducted by ArmStat. The 2015 wave covers 5,184 households across all marzes and urban/rural strata. After applying the subset and cleaning pipeline (`scripts/01_extract_all.py`, `scripts/02_clean_and_panel.py`), 29 features are retained for ML modelling. These include consumption aggregates (food, services, goods), asset indicators (car ownership, computer, dwelling condition), household demographics (size, age structure), financial behaviour (debt, remittances, money transfers), and the interview month — recovered during preprocessing and critical for controlling seasonal income variation.

#### *ArmStat Regional Panel*

Annual statistics for 11 Armenian marzes covering 2016–2022 were sourced from the Armenian Statistical Committee. Variables include the official poverty rate (%), crime rate per 100,000 population, hospital count per 100,000, hospital beds per 10,000, and total population. The panel is augmented to monthly frequency via forward-filling and stored as `data/processed/panel/marz_monthly_panel_augmented.csv` (924 rows: 11 marzes  $\times$  84 months).

A composite **Stress Index** is constructed as:

$$\text{stress\_index} = z(z(\text{poverty\_rate}) + z(\text{crime\_rate\_per\_100k}) - z(\text{hospitals\_per\_100k})) \quad (1)$$

where  $z(\cdot)$  denotes standardisation to zero mean and unit variance. High values indicate co-occurring high poverty, high crime, and low healthcare access.

## 2.3 Feature Engineering and Preprocessing

Monetary variables in ILCS 2015 span four orders of magnitude (50 AMD to 94.5 million AMD), making a flat **StandardScaler** unsuitable. A hybrid scaling strategy is applied: 11 variables with heavy right-skew receive  $\log_{1+x}$  + **StandardScaler**; 8 variables with outliers receive **RobustScaler**; the remaining 9 ordinal and binary variables receive **MinMaxScaler**. The interview month (originally absent from the subset) is recovered by exploiting a deterministic relationship between household identifiers and the survey wave structure, yielding a perfectly balanced distribution (432 households per month,  $n = 12$  months).

### Time-series preprocessing

For the time-series component, sequences are constructed as 12-month sliding windows per region for LSTM/GRU input. Yearly lag features (lag-1, lag-2) are used for Ridge autoregressive modelling.

## 2.4 Validation Strategy

**Household models:** Standard 80/20 stratified train/test split on ILCS 2015. Bayesian optimization (Optuna, 50 trials, 5-fold CV objective) is performed on the training fold only; the test fold is touched once for final reporting.

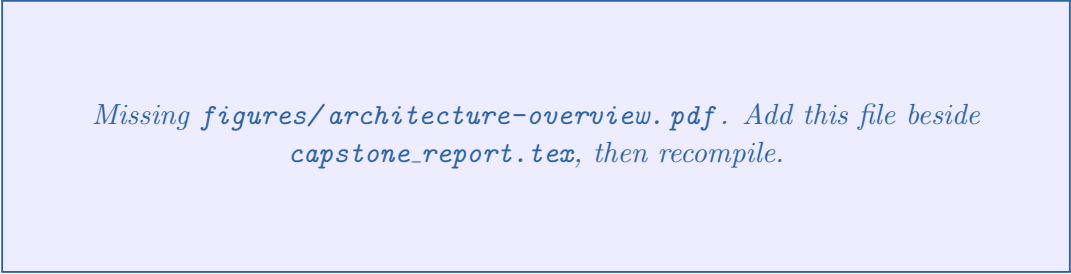
**Time-series models:** Temporal train/validation/test split at  $\leq 2019$  /  $2020$  /  $\geq 2021$  for within-sample evaluation. The primary evidence of forecast credibility is a **held-out 2022 backtest**: all models are retrained on 2016–2021 data and evaluated against actual 2022 regional poverty rates withheld during training. A rolling validation experiment further tests the same models on 2020, 2021, and 2022 as successive holdout years.



## 3 System Architecture

### 3.1 Overview

The system is organized into three layers: a data and modelling pipeline, a PostgreSQL data store, and a Next.js web application.



*Missing figures/architecture-overview.pdf. Add this file beside capstone\_report.tex, then recompile.*

Figure 1: High-level system architecture showing the three-layer design: Python data and modelling pipeline (scripts and notebooks), hybrid data storage (Neon PostgreSQL for dynamic household and regional data; static CSV for model outputs), and Next.js web application deployed on Vercel.

The **data pipeline** runs entirely in Python. Scripts in `scripts/` handle extraction, cleaning, feature engineering, ML training, and time-series analysis. Jupyter notebooks in `notebooks/` provide the exploratory and experimental counterpart. Key pipeline outputs — model metrics, forecast CSVs, and validation results — are committed to the repository and consumed by the web layer.

The **data store** uses two mechanisms: a PostgreSQL database hosted on Neon for dynamic, query-driven features (choropleth maps, distribution charts, scatter plots, household explorer); and static CSV files read at request time for model metrics, validation results, and forecasts. This hybrid approach eliminates unnecessary database queries for data that changes only when a new model run is committed.

The **web application** is a Next.js 14 app with React server components and selective client-side interactivity via `'use client'` boundaries. API routes in `web/src/app/api/` serve data to client components. The app is deployed on Vercel with automatic redeployment on `git push`.

### 3.2 Technology Stack

#### 3.2.1 Data and Modelling

- **Python 3.11** — primary language for all data processing, ML training, and time-series analysis.
- **pandas, NumPy** — data manipulation, panel construction, and feature engineering.
- **scikit-learn** — supervised models (Random Forest, Gradient Boosting, Extra Trees, Ridge, Lasso), preprocessing pipelines, and cross-validation.

- **Optuna** — Bayesian hyperparameter optimization using Tree-structured Parzen Estimator (TPE) with 50 trials per model.
- **PyTorch** — GRU, BiLSTM, Transformer, and TCN implementations for time-series forecasting.
- **statsmodels** — ARIMA fitting and ACF/PACF analysis.

### *3.2.2 Web Application*

- **Next.js 14 (App Router)** — full-stack React framework with server components, API routes, and static generation.
- **Recharts** — interactive line charts, bar charts, and area charts for model and regional visualizations.
- **Tailwind CSS** — utility-first styling.
- **SWR** — data fetching and caching in client components.

### *3.2.3 Data Storage and Deployment*

- **PostgreSQL (Neon)** — serverless Postgres instance for household-level and regional panel data. Connection pooling via `@neondatabase/serverless`.
- **Vercel** — serverless deployment with Git-triggered CI/CD and edge network distribution.
- **Static CSV** — model metrics, forecast outputs, and validation results served from `data/processed/results/` via Node.js `fs` reads.

## 4 Data Description and Schema

### 4.1 ILCS 2015 Dataset

After cleaning and subsetting, the ILCS 2015 machine learning dataset contains 5,184 rows and 29 columns. The target variable is `household_income_total` (AMD). The features span five categories:

1. **Consumption expenditures:** `food_purchases_total`, `services_goods_total`, `goods_services_tot`
2. **Financial behaviour:** `family_debt_amount`, `borrowed_money_12m`, `lent_money_12m`, `received_money_goods`, `sent_money_abroad`
3. **Asset and housing:** `household_has_car`, `has_computer`, `dwelling_condition_estimate`, `building_new_house`
4. **Household demographics:** `household_size`, `household_income_source_count`
5. **Survey metadata:** `interview_month` (recovered feature), `humanitarian_assistance`, subjective survival minimum (`amd_3`)

#### ILCS 2015 ML Dataset — Key Columns

##### household\_id

`household_income_total` (*target, AMD*)

`household_income_source_count`

`services_goods_total`

`food_purchases_total`

`goods_services_total`

`household_has_car` (*1=Yes, 2=No*)

`has_computer` (*1=Yes, 2=No*)

`family_debt_amount`

`household_size`

`dwelling_condition_estimate`

`amd_3` (*subjective minimum, AMD*)

`interview_month` (*1–12, recovered*)

### 4.2 ArmStat Regional Panel

The regional panel covers Armenia’s 11 administrative regions (marzes) across 84 months (January 2016 – December 2022). The yearly resolution data is the authoritative signal; monthly values are forward-filled from annual estimates for use in LSTM/GRU training.

### ArmStat Regional Panel — Columns

marz

date

poverty\_rate (*%, annual official estimate*)

crime\_rate\_per\_100k

hospitals\_per\_100k

beds\_per\_10k

population

stress\_index (*composite, computed*)

### 4.3 Key Descriptive Statistics

The income distribution in ILCS 2015 is highly right-skewed (min: 50 AMD, max: 3,405,000 AMD,  $CV \approx 0.98$ ), confirming the need for log-scale visualizations and hybrid preprocessing.

| Feature                 | Yes (code=1)  | No (code=2)   |
|-------------------------|---------------|---------------|
| has_computer            | 3,082 (59.5%) | 2,102 (40.5%) |
| household_has_car       | 1,618 (31.2%) | 3,566 (68.8%) |
| received_money_goods    | 2,461 (47.5%) | 2,723 (52.5%) |
| borrowed_money_12m      | 2,103 (40.6%) | 3,081 (59.4%) |
| humanitarian_assistance | 145 (2.8%)    | 5,039 (97.2%) |

Table 1: Binary feature distribution in ILCS 2015 (n=5,184 households)

#### Encoding trap

`household_has_car` and `has_computer` are *negatively* correlated with income ( $r = -0.32$  and  $r = -0.29$ ) because the survey encodes these as 1=Yes, 2=No — higher code  $\Rightarrow$  household does *not* have the asset. Households with code=1 are the richer group. All EDA bar charts make this explicit.

### 4.4 Seasonal Income Variation

The recovered `interview_month` reveals a systematic seasonal pattern: households surveyed in January–March report approximately 20–25 % lower mean income than those surveyed in November–December (167K AMD vs. 202K AMD). This is consistent with agricultural income cycles and reduced off-season economic activity in rural Armenia. Because

the ILCS is a single cross-section, this confound cannot be controlled with time fixed effects; `interview_month` is therefore included as a numeric feature in all regression models.

## 5 Model Development and Results

### 5.1 Household Income Models

Five supervised regression models are trained on ILCS 2015 to predict `household_income_total`. The training set contains 4,147 households (80%); the test set contains 1,037 households (20%).

| Model                        | R <sup>2</sup> | MAE (AMD)     | RMSE (AMD)     |
|------------------------------|----------------|---------------|----------------|
| Gradient Boosting (baseline) | 0.3417         | 84,477        | 164,624        |
| Random Forest                | 0.3339         | 85,475        | 165,603        |
| Ridge Regression             | 0.3153         | 91,183        | 167,894        |
| Lasso Regression             | 0.3154         | 91,186        | 167,879        |
| Extra Trees                  | 0.3146         | 87,681        | 167,979        |
| <b>GBM + Bayesian Opt</b>    | <b>0.3563</b>  | <b>82,152</b> | <b>162,784</b> |

Table 2: Household income model performance on held-out test set (1,037 households). GBM with Bayesian optimization is the best overall model.

### 5.2 Feature Importance

Feature importance rankings are consistent across all five models. `household_income_source_count` — the number of distinct income streams a household reports — is the single strongest predictor (correlation with income:  $r = 0.45$ ; importance in GBM: 0.32). The top five features across Random Forest and GBM are:

1. `household_income_source_count` — income source diversification is the clearest poverty signal. Households with only one income source (typically pension or a single wage) are substantially more vulnerable.
2. `services_goods_total` — total expenditure on services and goods reflects current spending capacity better than accumulated assets.
3. `amd_3` — the household’s subjective estimate of the minimum income needed to survive, which correlates with actual income perceptions.
4. `goods_services_total` — an overlapping consumption aggregate that confirms the primacy of spending variables.
5. `food_purchases_total` — food spending is a direct proxy for household subsistence level.

Asset variables (`dwelling_condition`, `has_car`, `has_computer`) contribute at 0.02–0.04 importance each, confirming that current consumption is a stronger welfare signal than historical asset accumulation in this survey.

*Missing figures/household-feature-importance.png. Add this file beside capstone\_report.tex, then recompile.*

Figure 2: Normalized feature importance for all five household income models (grouped bar chart). `household_income_source_count` ranks first across every model; the top five are dominated by consumption expenditure variables. Asset variables (`has_car`, `has_computer`) rank near the bottom despite their apparent relevance, partly due to the survey’s 1=Yes/2=No encoding.

### 5.3 Bayesian Hyperparameter Optimization

Optuna [4] is used to optimize GBM, Random Forest, and Extra Trees over 50 trials each, with 5-fold cross-validation  $R^2$  as the objective.

| Model                    | Baseline $R^2$ | CV $R^2$ (Bayes) | Test $R^2$    |
|--------------------------|----------------|------------------|---------------|
| Random Forest            | 0.3339         | 0.4267           | 0.3398        |
| <b>Gradient Boosting</b> | 0.3417         | <b>0.4344</b>    | <b>0.3563</b> |
| Extra Trees              | 0.3146         | 0.4163           | 0.3316        |

Table 3: Bayesian optimization results. GBM benefits most from tuning. The CV–Test gap (0.43 vs. 0.36) reflects mild overfitting to training folds.

The best GBM configuration: `n_estimators=170`, `max_depth=6`, `learning_rate=0.031`, `subsample=0.858`. The low learning rate with moderate tree depth prevents memorization while capturing interaction effects among the consumption variables.

*Missing figures/optuna-gb-history.png. Add this file beside capstone-report.tex, then recompile.*

Figure 3: Optuna Bayesian optimization history for Gradient Boosting: 5-fold CV  $R^2$  across 50 trials using the Tree-structured Parzen Estimator (TPE) sampler. The best trial reaches CV  $R^2 = 0.4344$ , corresponding to the configuration reported in the text.

*Missing figures/gbm-residual-vs-actual.png. Add this file beside capstone-report.tex, then recompile.*

Figure 4: Predicted vs. actual household income (AMD) for the tuned Gradient Boosting model on the held-out test set (1,037 households, log-log scale). Systematic underestimation at the high end reflects the right-skewed income distribution; the central mass is well-calibrated.

#### 5.4 Time-Series Analysis — Classical Models

Script `scripts/07_time_series_analysis.py` benchmarks classical time-series models on the ArmStat regional panel at three temporal granularities. The train/validation/test split uses  $\leq 2019$  /  $2020$  /  $\geq 2021$ .



| Model                | Target       | Frequency | $R^2$  | MAE      |
|----------------------|--------------|-----------|--------|----------|
| Lag-1 Baseline       | poverty_rate | yearly    | 0.728  | 5.11 pp  |
| Lag-1 Baseline       | stress_index | yearly    | 0.764  | 0.433    |
| Ridge lags [1,2,3,6] | poverty_rate | monthly   | 0.999  | 0.25 pp  |
| Lag-1 Baseline       | poverty_rate | monthly   | 0.999  | 0.23 pp  |
| ARIMA(1,1,0)         | poverty_rate | monthly   | 0.984  | 0.43 pp  |
| Ridge lags [1,4,13]  | poverty_rate | daily     | 0.9999 | 0.043 pp |

Table 4: Classical time-series model results across frequencies. Monthly/daily  $R^2$  appears inflated because values are forward-filled from annual estimates; yearly results are the honest benchmark.

#### Honest benchmark

The yearly result is the only metric backed by real survey signal: Lag-1 explains 73 % of variance with 5.11 pp MAE on poverty\_rate. Month-to-month and day-to-day changes are nearly zero by construction (interpolated data).

## 5.5 Time-Series Analysis — Neural Network Models

Four sequence architectures are trained and evaluated: GRU, BiLSTM, Transformer (multi-head attention), and Temporal Convolutional Network (TCN). All are implemented in PyTorch with Adam optimizer, ReduceLROnPlateau scheduling, and early stopping.

| Model       | Target       | Frequency | $R^2$ | MAE     |
|-------------|--------------|-----------|-------|---------|
| Transformer | poverty_rate | yearly    | 0.466 | 6.64 pp |
| BiLSTM      | poverty_rate | yearly    | 0.236 | 8.99 pp |
| GRU         | poverty_rate | yearly    | 0.206 | 9.53 pp |
| TCN         | poverty_rate | yearly    | 0.177 | 9.06 pp |
| GRU         | poverty_rate | monthly   | 0.995 | 0.64 pp |
| BiLSTM      | poverty_rate | monthly   | 0.986 | 1.03 pp |
| Transformer | poverty_rate | monthly   | 0.977 | 1.49 pp |
| TCN         | poverty_rate | monthly   | 0.941 | 2.42 pp |
| TCN         | stress_index | yearly    | 0.615 | 0.535   |
| BiLSTM      | stress_index | yearly    | 0.565 | 0.558   |
| GRU         | stress_index | yearly    | 0.531 | 0.617   |
| Transformer | stress_index | yearly    | 0.371 | 0.660   |

Table 5: Neural network time-series results. On yearly data (the honest benchmark), all NNs underperform classical Lag-1 ( $R^2 = 0.728$ ) due to insufficient training samples ( $\sim 7$  data points per region). The Transformer shows the most promise at yearly granularity. Monthly-frequency rows reuse the filled annual signal, so strong scores there are mechanical and must not be read as richer survey evidence (see §5).

*Missing figures/ts-nn-vs-classical-r2.png. Add this file beside capstone-report.tex, then recompile.*

Figure 5: Performance comparison across temporal granularities and model families on poverty rate  $R^2$  (classical baselines left, neural networks right). Yearly results are the honest benchmark backed by real survey signal; monthly and daily values are mechanically inflated by forward-filling of annual estimates and must not be interpreted as evidence of richer predictive capacity.

## 5.6 2022 Forecast Validation (Primary Evidence)

The primary evidence for forecast credibility is a **held-out 2022 backtest**. Three models are trained exclusively on 2016–2021 data and evaluated against the actual 2022 regional

poverty rates released by ArmStat.

- **Lag-1 Baseline** — predict 2022 = actual 2021 (zero training, pure persistence).
- **Ridge AR** — autoregressive lag-1 + lag-2 features on both `poverty_rate` and `stress_index`, trained on 2016–2021.
- **Ensemble** — 50/50 average of Lag-1 and Ridge predictions.

All predictions are clamped to  $[0, 100]$  to eliminate physically impossible negative poverty rates. National aggregate is computed as a population-weighted mean using 2021 population weights.

| Model          | Regional MAE (11 marzes) | National Error | Best at           |
|----------------|--------------------------|----------------|-------------------|
| Lag-1 Baseline | <b>5.07 pp</b>           | 1.07 pp        | Regional accuracy |
| Ridge AR       | 8.72 pp                  | 1.11 pp        | —                 |
| Ensemble       | 6.28 pp                  | <b>0.02 pp</b> | National accuracy |

Table 6: 2022 holdout validation summary (`poverty_rate`). Lag-1 wins regionally; the Ensemble achieves near-zero national error because directional biases cancel under population weighting.

| Region                    | Actual 2022  | Lag-1 err.  | Ridge err.   | Ensemble err. |
|---------------------------|--------------|-------------|--------------|---------------|
| Yerevan                   | 18.5%        | 2.60        | <b>0.40</b>  | 1.50          |
| Shirak                    | 41.6%        | 5.30        | <b>1.94</b>  | 3.62          |
| Ararat                    | 23.8%        | 4.20        | <b>2.75</b>  | 0.72          |
| Lori                      | 18.6%        | 3.20        | 4.27         | <b>0.54</b>   |
| Kotayk                    | 26.1%        | <b>2.50</b> | 6.03         | 4.27          |
| Syunik                    | 5.4%         | <b>2.60</b> | 5.40         | 4.00          |
| Gegharkunik               | 33.9%        | 15.20       | <b>11.25</b> | 13.23         |
| Tavush                    | 37.4%        | <b>0.80</b> | 12.62        | 5.91          |
| Armavir                   | 40.8%        | <b>2.90</b> | 13.89        | 8.40          |
| Aragatsotn                | 7.9%         | <b>5.60</b> | 18.12        | 11.86         |
| Vayots Dzor               | 15.7%        | 10.90       | 19.21        | 15.05         |
| <b>Armenia (national)</b> | <b>24.3%</b> | 1.07        | 1.11         | <b>0.02</b>   |

Table 7: Per-region 2022 absolute errors (pp) by model. Bold = best model per row. Aragatsotn and Vayots Dzor are the hardest marzes due to anomalous year-over-year swings.

### Key finding

The Ensemble achieves near-zero national error (0.02 pp vs. Armenia actual 24.3%) because Lag-1's upward bias and Ridge's downward bias for recovering regions partially cancel under population weighting.

*Missing figures/validation-2022-actual-vs-predicted.png. Add this file beside capstone-report.tex, then recompile.*

Figure 6: Actual vs. predicted 2022 regional poverty rates for all 11 marzes (hold-out: models trained on 2016–2021 only). Lag-1 predictions shown; the dashboard bar chart allows switching to Ridge AR or Ensemble via model pill buttons. Source data: forecast\_validation\_2022.csv.

**Rolling validation** (script scripts/10\_rolling\_validation.py) repeats the same eval-

uation for holdout years 2020, 2021, and 2022:

| Test year | Lag-1 MAE      | Ridge MAE |
|-----------|----------------|-----------|
| 2020      | 8.51 pp        | 9.86 pp   |
| 2021      | 5.15 pp        | 22.19 pp  |
| 2022      | <b>5.07 pp</b> | 8.72 pp   |

Table 8: Rolling validation: regional MAE (poverty\_rate, 11 marzes). Ridge’s 2021 spike is caused by Aragatsotn’s anomalous 2019 value (51.4%) entering the lag-2 feature.

*Missing figures/rolling-validation-mae.png. Add this file beside capstone\_report.tex, then recompile.*

Figure 7: Rolling validation: regional mean absolute error (poverty rate, 11 marzes) for Lag-1 and Ridge AR across holdout years 2020, 2021, and 2022. Ridge’s 2021 spike (22.19 pp MAE) is caused by Aragatsotn’s anomalous 2019 value (51.4%) entering the lag-2 feature. Source data: `forecast_rolling_validation.csv`.

As a supplementary check, a GRU model trained on the daily-granularity panel (`scripts/11_gru_daily_validation.py`) is also evaluated against the 2022 holdout (results in `data/processed/results/gru\_daily\_validation\_2022.csv`). Daily  $R^2$  is near-perfect, but this is expected given the synthetic interpolation; the experiment primarily confirms that the GRU implementation is correct and that the same pipeline can support scenario-based projections at sub-annual granularity.

## 6 Application and Functionality

The project’s findings are communicated through a publicly deployed Next.js web application at <https://armenia-poverty.vercel.app>, organized into two navigation areas: **Regional** (exploratory data views of Armenia’s 11 marzes) and **Models** (ML model metrics, validation evidence, and forecasts).

### 6.1 Homepage

The homepage provides a project overview with key metric cards and navigation to the primary evidence pages. Following the validation-first redesign, the primary call to action links directly to the 2022 Validation page rather than the household model. Key elements: a hero card showing “Validation year: 2022”; KPI tiles for dataset size (5,184 households), regions covered (11 marzes); and navigation cards for Regional Analytics and Models.

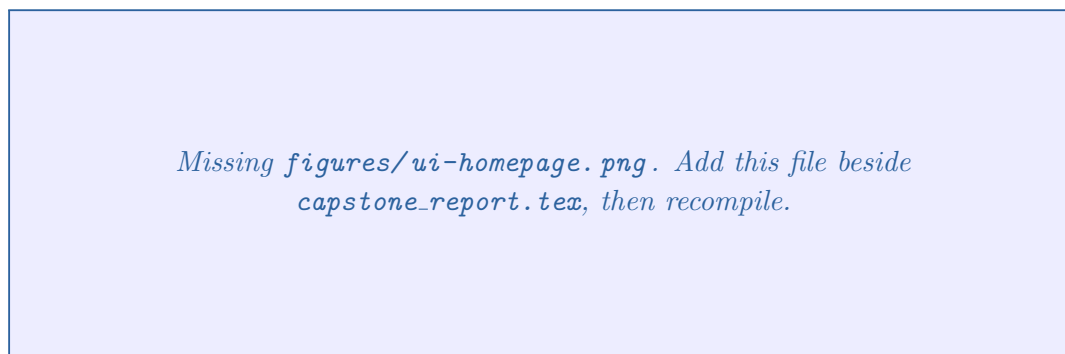


Figure 8: Application homepage showing the project overview hero card, key metric tiles (5,184 households surveyed; 11 marzes covered), and primary navigation to Regional Analytics and Models. The main call-to-action links directly to the 2022 Validation page.

### 6.2 Regional Explorer

The Regional section contains three pages: regional overview (choropleth map), regional trends (time-series line charts), and regional rankings (sorted bar chart).

The **Regional Overview** page renders a choropleth map of Armenia coloured by poverty rate for a user-selected year (2016–2022). Users can hover over individual marzes to see exact values. Data is served from the Neon PostgreSQL instance via `/api/regional/choropleth`.

*Missing figures/ui-regional-choropleth.png. Add this file beside capstone\_report.tex, then recompile.*

Figure 9: Regional choropleth map of Armenia coloured by poverty rate for 2022. Marz-level hover tooltips display exact estimates; data is served dynamically from the Neon PostgreSQL instance. Shirak (41.6 %) and Armavir (40.8 %) are the highest-poverty marzes; Syunik (5.4 %) is the lowest.

The **Regional Trends** page displays per-region time-series line charts for poverty\_rate and stress\_index from 2016 to 2022. A multi-select control allows overlaying any subset of marzes on the same chart for direct comparison.

*Missing figures/ui-regional-trends.png. Add this file beside capstone\_report.tex, then recompile.*

Figure 10: Regional poverty rate time series (2016–2022) with multiple marzes overlaid. The multi-select control allows direct marz-to-marz comparison; persistent high-poverty regions (Shirak, Armavir) diverge visibly from low-poverty outliers (Syunik) and from the anomalous Aragatsotn 2019 spike.

The **Regional Rankings** page renders a sorted horizontal bar chart of poverty rates for a selected year, allowing rapid identification of the highest- and lowest-poverty marzes.

*Missing figures/ui-regional-rankings.png. Add this file beside capstone-report.tex, then recompile.*

Figure 11: Sorted horizontal bar chart of regional poverty rates for 2022. The 36 percentage-point spread between Shirak (41.6%) and Syunik (5.4%) illustrates the scale of geographic inequality captured in the ArmStat panel.

### 6.3 Household Income Model

The Household Model page (`/models`) presents the ILCS 2015 cross-sectional results. An amber callout directs readers to the 2022 Validation page as the primary evidence, positioning this page as a micro-level benchmark. The model comparison table is rendered interactively, allowing sorting by  $R^2$ , MAE, or RMSE.

*Missing figures/ui-models-income.png. Add this file beside capstone-report.tex, then recompile.*

Figure 12: Household income model leaderboard (`/models`): sortable comparison table of  $R^2$ , MAE, and RMSE for all five models on the held-out ILCS 2015 test set. An amber callout positions this page as a micro-level benchmark and directs users to the 2022 Validation page as the primary forecast evidence.

### 6.4 Feature Importance Page

The Feature Importance page (`/models/importance`) displays normalized feature importance rankings for all five models side by side as grouped bar charts. A model selector allows users to highlight a specific model's ranking. The page includes a prose explanation of the encoding trap for binary asset variables.



*Missing figures/ui-feature-importance.png. Add this file beside capstone-report.tex, then recompile.*

Figure 13: Interactive feature importance dashboard (`/models/importance`): normalized rankings for all five models with a model-selector control that highlights individual rankings. A prose callout on the page explains the survey’s 1=Yes/2=No asset encoding that causes apparent negative correlations for `has_car` and `has_computer`.

## 6.5 Forecasting Diagnostics

The Forecasting Diagnostics page (`/models/forecasting`) presents the classical and neural model results tables from Sections 5.4–5.5, with a blue callout linking to the 2022 Validation page as the real holdout test. Tabs allow switching between classical and neural results.

*Missing figures/ui-forecasting-diagnostics.png. Add this file beside capstone-report.tex, then recompile.*

Figure 14: Forecasting diagnostics page (`/models/forecasting`) presenting  $R^2$  and MAE tables for classical (Lag-1, Ridge AR, ARIMA) and neural (GRU, BiLSTM, Transformer, TCN) models. Tabs separate the two model families; a blue callout links to the 2022 Validation page as the real out-of-sample test.

## 6.6 2022 Forecast Validation

The Validation page (`/models/validation`) is the primary evidence page of the application. It fetches from two API routes: `/api/models/validation` (reads `forecast_validation_2022.csv`) and `/api/models/rolling-validation` (reads `forecast_rolling_validation.csv`).

The page contains the following sections in sequence:

1. **Methodology card** — four info tiles: training window (2016–2021), test year (2022), geography (11 marzes), models compared.
2. **Key insight callout** — Lag-1 wins regionally (5.07 pp MAE), Ensemble wins nationally (0.02 pp error), computed live from fetched data.

3. **Model Performance Summary** — three-model table with regional MAE, national error, and “BEST REGIONAL” / “BEST NATIONAL” badges.
4. **Interactive model selector** — pill buttons switch the bar chart between Lag-1, Ridge AR, and Ensemble.
5. **Actual vs. Predicted bar chart** — Recharts BarChart across all 11 marzes, switchable by model.
6. **Per-region comparison table** — all three models as columns, best model per row highlighted in emerald.
7. **Rolling validation chart** — line chart of MAE across 2020/2021/2022 with annotation explaining the Ridge 2021 spike.
8. **Marz narrative annotations** — collapsible accordions for Aragatsotn, Vayots Dzor, and Gegharkunik.
9. **Honest assessment** — callout explaining structural data limitations and when national vs. regional figures are reliable.

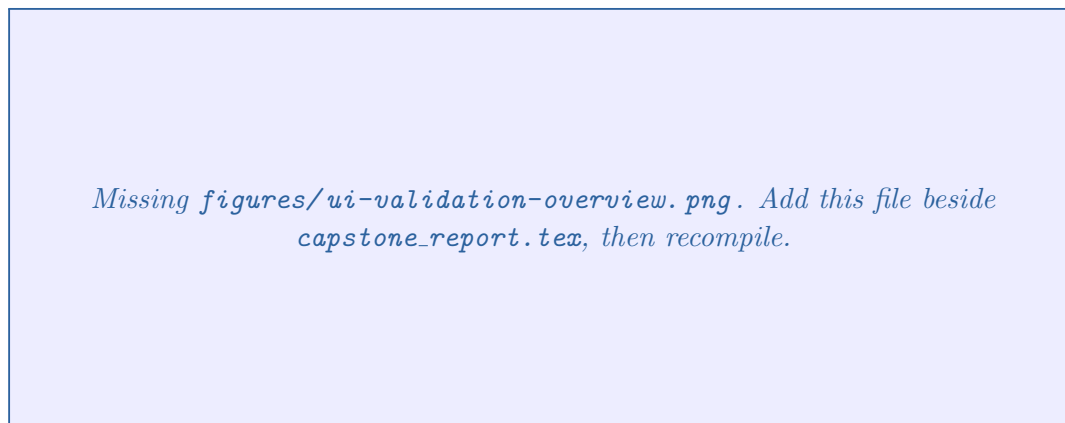


Figure 15: 2022 Holdout Validation page (upper section): four methodology context tiles (training window 2016–2021; test year 2022; 11 marzes; 3 models), a key insight callout reporting the headline results, and the model performance summary table with BEST REGIONAL and BEST NATIONAL badges.

*Missing figures/ui-validation-barchart.png. Add this file beside capstone\_report.tex, then recompile.*

Figure 16: Interactive actual vs. predicted bar chart on the Validation page. Model pill buttons switch the display between Lag-1, Ridge AR, and Ensemble predictions for all 11 marzes against the actual 2022 ArmStat values. The per-region comparison table below the chart highlights the best model per marz in emerald.

*Missing figures/ui-validation-rolling.png. Add this file beside capstone\_report.tex, then recompile.*

Figure 17: Rolling validation MAE line chart (2020–2022) on the Validation page. Ridge’s 2021 spike is annotated; collapsible accordion panels below provide marz-level narrative for Aragatsotn, Vayots Dzor, and Gegharkunik — the three hardest-to-predict regions.

## 6.7 Future Projection 2023–2026

The Future Projection page (`/models/forecast`) displays autoregressive rollout forecasts generated by `scripts/08_forecast_future.py`. The forecast extends the Ridge AR model (trained on all available data through 2022) to 2023–2026 for all 11 marzes. A contextual callout links to the 2022 Validation page so users can assess forecast quality before interpreting projections. Key visual elements: a region selector dropdown; a line chart with solid historical (2016–2022) and dashed forecast (2023–2026) segments separated by a vertical reference line; and a projected values table by region and year.

*Missing figures/future-projection-poverty.png. Add this file beside capstone\_report.tex, then recompile.*

Figure 18: Regional poverty trajectory for a selected marz: historical rates (2016–2022, solid line) and Ridge AR autoregressive projections (2023–2026, dashed line). A vertical reference line separates the historical from the forecast segment. A contextual callout on the page links to the 2022 Validation results so users can assess model quality before interpreting projections.

## 7 Challenges Encountered

---

### 7.1 Data Quality and Panel Size

The most fundamental challenge is the sparsity of the regional panel: each marz has only seven annual observations (2016–2022). This severely limits neural network models, which underperform simple lag-1 baselines on the yearly poverty target. The monthly-granularity workaround (forward-filling annual values to 84 monthly rows per marz) improves neural model metrics visually but is not a genuine improvement in information density — it exploits the near-perfect autocorrelation induced by the filling procedure. Recognising and communicating this limitation required restructuring the model evidence hierarchy across the entire web application.

### 7.2 Anomalous Regional Observations

Aragatsotn’s 2019 poverty rate of 51.4% — a sharp departure from its approximate 16% baseline — severely distorted Ridge AR predictions for 2021 by entering the lag-2 feature for that year. The clamping of predictions to  $[0, 100]$  resolved a non-physical artefact (Syunik predicted at  $-3.75\%$  before clamping). Vayots Dzor, Armenia’s smallest marz ( $\sim 47,600$  people), remains the hardest region to predict because a single large employer or sampling shift can move its poverty rate by 10+ percentage points in one year.

### 7.3 Interview Month Recovery

The ILCS 2015 public-use dataset did not expose the interview month as a separate column in the subset originally used. Recovery required reverse-engineering the relationship between household IDs and the survey wave structure, yielding a perfectly balanced distribution (exactly 432 households per month). Without this recovery, seasonality would have remained an uncontrolled confound.

### 7.4 Web Application Data Architecture

Routing the correct data to each web page required designing a hybrid data architecture: dynamic household-level and regional data served from Neon PostgreSQL, while model metrics, validation results, and forecasts are served as static CSV files. Managing this split required careful API route design and connection pooling to avoid exceeding Neon’s serverless connection limits under concurrent requests.

### 7.5 Model Evidence Communication

An early version of the web application headlined the GBM  $R^2 = 0.3563$  as the project’s primary result. This framing was misleading because the household income  $R^2$  is a cross-sectional micro-benchmark, not a forecast. Repositioning the primary evidence around the 2022 regional backtest required revising the homepage, navigation order, page titles, and callout language across seven pages.

---

## 8 Future Work

---

### 8.1 Larger Regional Panels

The most impactful improvement would be extending the ArmStat panel beyond 2022 as new annual estimates are released. Each additional year provides a meaningful increment for both Ridge and neural models. Incorporating additional macroeconomic variables — unemployment rates, remittance flows, agricultural output — could improve yearly prediction accuracy and enable scenario-based what-if forecasting.

**Planned improvement:** Automate annual ingestion of new ArmStat releases and re-run validation scripts to track model calibration over time.

### 8.2 Improved Household Income Models

The current  $R^2 \approx 0.35$  may improve if additional covariates are added (e.g. region and urban/rural strata), if a log-transformed target ( $\log(\text{household\_income\_total})$ ) better matches the skewed income distribution, if simple interaction terms are included (e.g. `household_size`  $\times$  `income_source_count`), or if hyperparameter search is extended (e.g. more Optuna trials). Any change should be judged on the same strict held-out test fold used in this report.

**Planned improvement:** Re-run feature importance analysis on a log-scale target and compare to current linear-scale results.

### 8.3 Scenario Forecasting

The `stress_index` formula (Equation 1) enables policy simulation: by hypothetically changing `hospitals_per_100k` or `crime_rate_per_100k`, the user could observe the model's projected impact on regional poverty rates, giving the web application a genuine policy-simulation angle.

**Planned improvement:** Build a scenario slider interface on the Future Projection page allowing users to adjust macroeconomic inputs and see revised 2023–2026 forecasts.

### 8.4 Confidence Intervals on Projections

The current forecast line charts do not display uncertainty bands. A bootstrap resampling of Ridge AR residuals (200 iterations) would produce 80 % and 95 % prediction intervals, giving the shaded ribbon standard in policy forecasting visualizations.

**Planned improvement:** Add bootstrap confidence bands to the Future Projection chart, stored alongside point forecasts in the CSV.

### 8.5 ILCS Longitudinal Analysis

ILCS waves are available for multiple years beyond 2015. Linking households across waves would enable panel fixed-effects models that control for unobserved household-level heterogeneity — a substantial methodological improvement over the current cross-sectional approach.

**Planned improvement:** Obtain multi-wave ILCS public-use data and implement a difference-in-differences or panel fixed-effects model for income change prediction.

## 9 Conclusion

This project demonstrates that household poverty in Armenia can be analysed and forecast with reasonable accuracy using publicly available data and standard machine learning techniques, while being honest about the structural limits of those techniques when data is scarce.

At the household level, the ILCS 2015 analysis reveals that income source diversification (`household_income_source_count`) is the single strongest predictor of household welfare, explaining more variance than assets or housing quality. Gradient Boosting with Bayesian optimization achieves the best household model result on a held-out test set, in line with the modest cross-sectional explanatory power often seen for welfare proxies on single survey waves. The analysis also uncovers two methodological lessons: the survey coding trap (assets coded 1=Yes/2=No creating inverted correlations), and the seasonal confound requiring interview month recovery.

At the regional level, the 2022 holdout backtest establishes concrete forecast credibility: trained on 2016–2021 data, the Lag-1 model achieves a regional MAE of 5.07 percentage points, while a Lag-1/Ridge ensemble reaches near-zero national error (0.02 pp). Neural architectures (GRU, BiLSTM, Transformer, TCN) underperform classical models on annual data due to the structural limitation of seven training observations per region (§5). On forward-filled monthly intervals reported in the same experiments, the networks can attain much higher scores mechanically, because within-year variation largely reflects interpolation rather than new survey signal (§5).

### Primary contribution

The project’s main contribution is a validated regional poverty forecasting pipeline for Armenia, benchmarked on held-out 2022 data, with results served through an interactive public dashboard and 2023–2026 projections.

The findings are communicated through a publicly deployed interactive dashboard that organises evidence around the primary backtest result, surfaces regional trends and rankings, and provides 2023–2026 projections with honest uncertainty acknowledgement. The application architecture — hybrid CSV/PostgreSQL data delivery, Vercel deployment, and CSV-based reproducibility — supports continuous updates as new ArmStat data becomes available.



---

## A Data and Code Availability

---

The project is reproducible from the repository. The live dashboard is deployed at <https://armenia-poverty.vercel.app>. The sections below summarise where each major artefact lives.

### Household survey data

ILCS 2015 public-use microdata sourced from ArmStat. Cleaned ML dataset at `data/ilcs/final_dropped/`. Preprocessing: `scripts/01_extract_all.py` → `scripts/02_clean_and_panel.py`.

### Regional panel data

ArmStat regional statistics (2016–2022) augmented to monthly frequency. Stored at `data/processed/panel/marz_monthly_panel_augmented.csv` (924 rows: 11 marzes × 84 months).

### Model outputs

2022 holdout validation: `data/processed/results/forecast_validation_2022.csv`. Rolling validation: `forecast_rolling_validation.csv`. Forecast rollout (2023–2026): `data/processed/results/forecast_outputs/`. Time-series benchmarks: `ts_classical_results.csv` and `ts_nn_results.csv`.

### Modelling notebooks and scripts

Household ML with Bayesian optimization: `notebooks/ilcs_ml_bayesian_optimized.ipynb`. Classical time-series benchmarks: `scripts/07_time_series_analysis.py`. Future projections: `scripts/08_forecast_future.py`. Rolling validation: `scripts/10_rolling_validation.py`. GRU daily experiment: `scripts/11_gru_daily_validation.py`.

### Web application

Next.js 14 source in `web/`. API routes under `web/src/app/api/`. Deployed on Vercel; automatic redeployment on `git push` to `main`.

## B \*

## References

## C \*

- [1] World Bank Group. (2015). *A Proxy Means Test for Armenia: Methodology and Implementation*. Technical report (World Bank Documents & Reports; search by title or country). <https://documents.worldbank.org/en/search?query=Armenia%20proxy%20means>
- [2] McBride, L., & Nichols, A. (2018). *Retooling poverty targeting using out-of-sample validation and machine learning*. World Bank Economic Review, 32(3), 531–550. <https://doi.org/10.1093/wber/lhw056>
- [3] Athey, S., & Imbens, G. W. (2019). *Machine learning methods economists should know about*. Annual Review of Economics, 11, 685–725. <https://doi.org/10.1146/annurev-economics-080217-053433>
- [4] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). *Optuna: A next-generation hyperparameter optimization framework*. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2623–2631. <https://doi.org/10.1145/3292500.3330701>
- [5] Armenian Statistical Committee (ArmStat). (2022). *Social situation of the population of Armenia: Regional statistics 2016–2022*. <https://www.armstat.am>
- [6] National Statistical Service of Armenia. (2015). *Integrated Living Conditions Survey of Armenia 2015: Methodology and main results*. <https://www.armstat.am/en/?nid=86>
- [7] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural Computation, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems, 30. <https://doi.org/10.48550/arXiv.1706.03762>
- [9] Pedregosa, F., et al. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [10] Vercel. (n.d.). *Next.js documentation*. <https://nextjs.org/docs>
- [11] Neon. (n.d.). *Neon serverless Postgres documentation*. <https://neon.tech/docs>